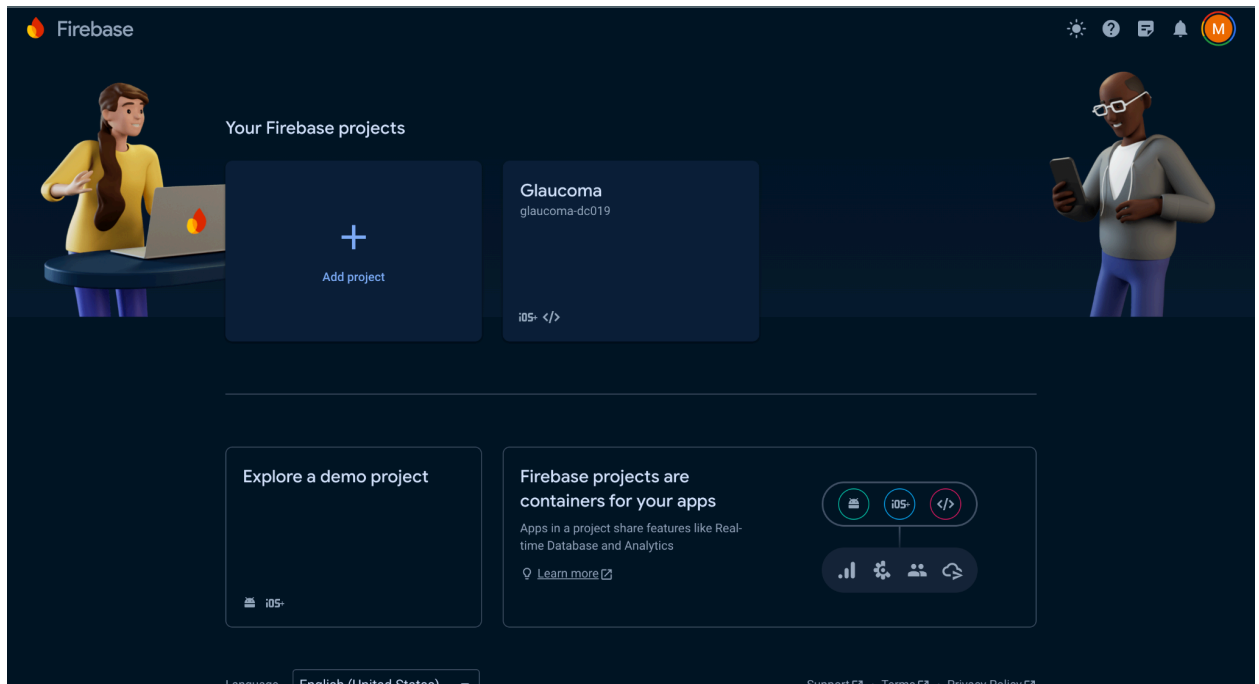


Integrating a New Firebase Database

1. Create new Firestore Database

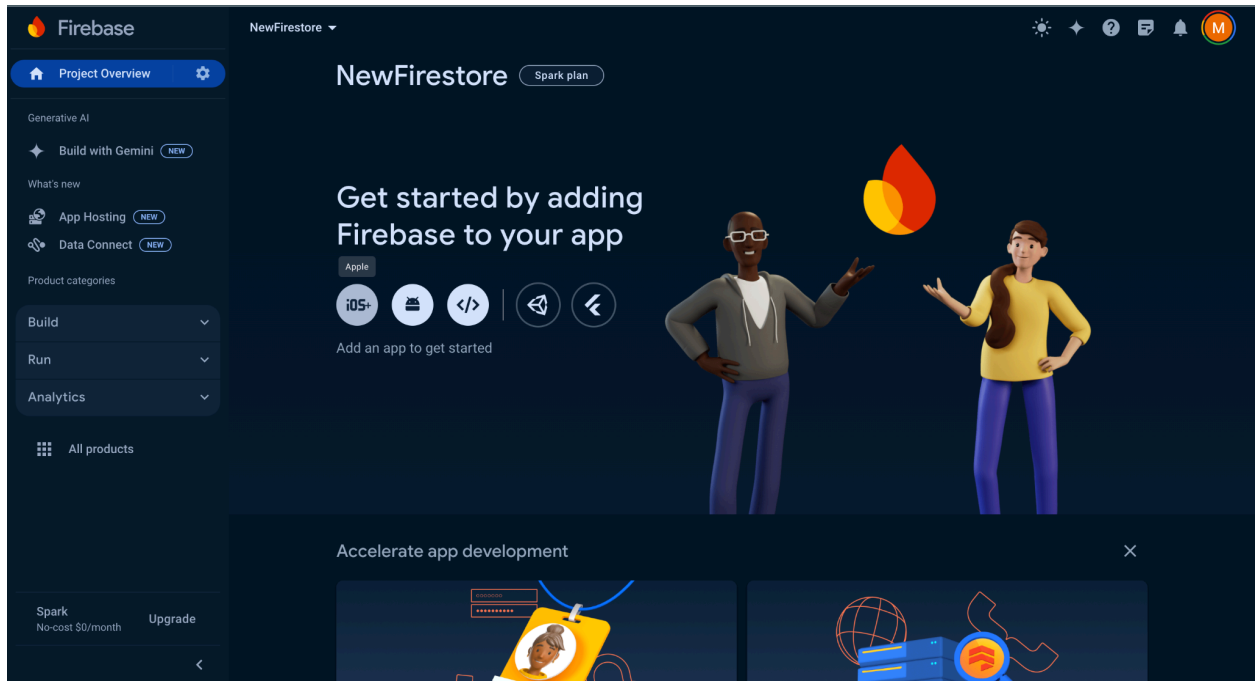
Go to <https://console.firebase.google.com/> and sign in with a .gmail account.
Then, click 'Add Project'



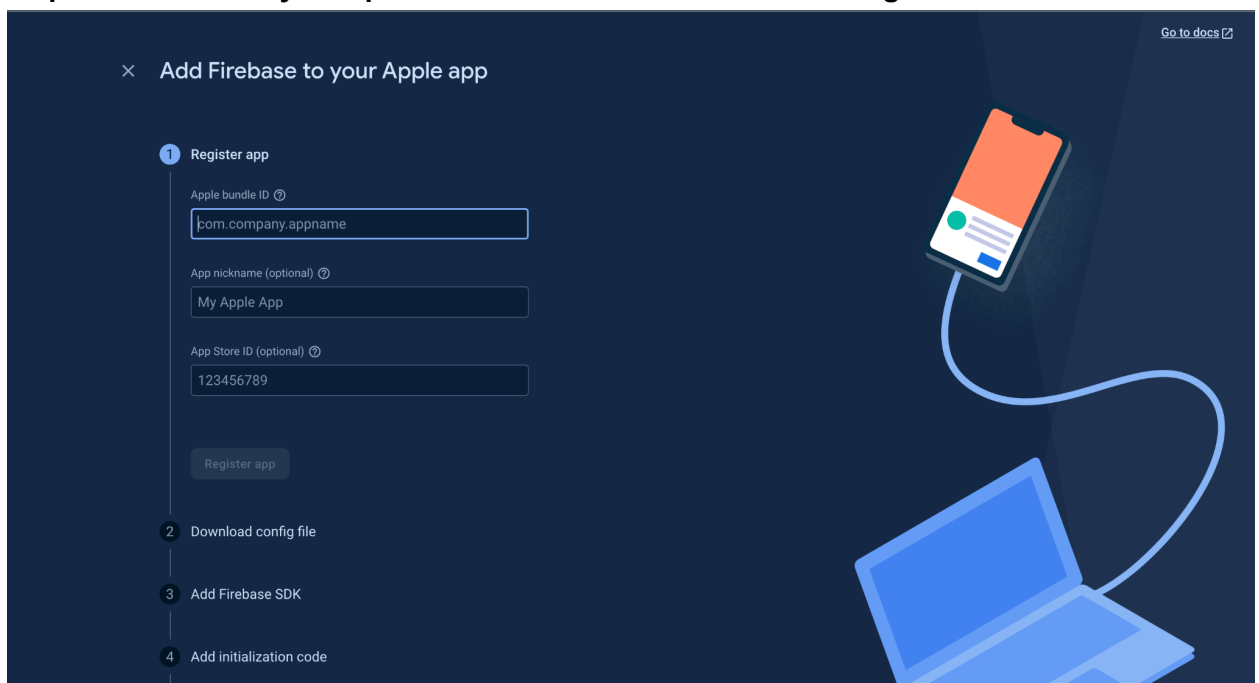
Continue with the setup using the default settings.

2. Connect the Firestore to the Swift Application

In the project overview, click on the iOS icon to access the



Follow the next steps to get the correct data lists that are compatible with the application. Note that you may have to reference the XCode settings to ensure the details match. Ensure that the .plist file is saved in step 2 and added to the repository as instructed! **Steps 3-5 are already completed and do NOT need to be done again!!**

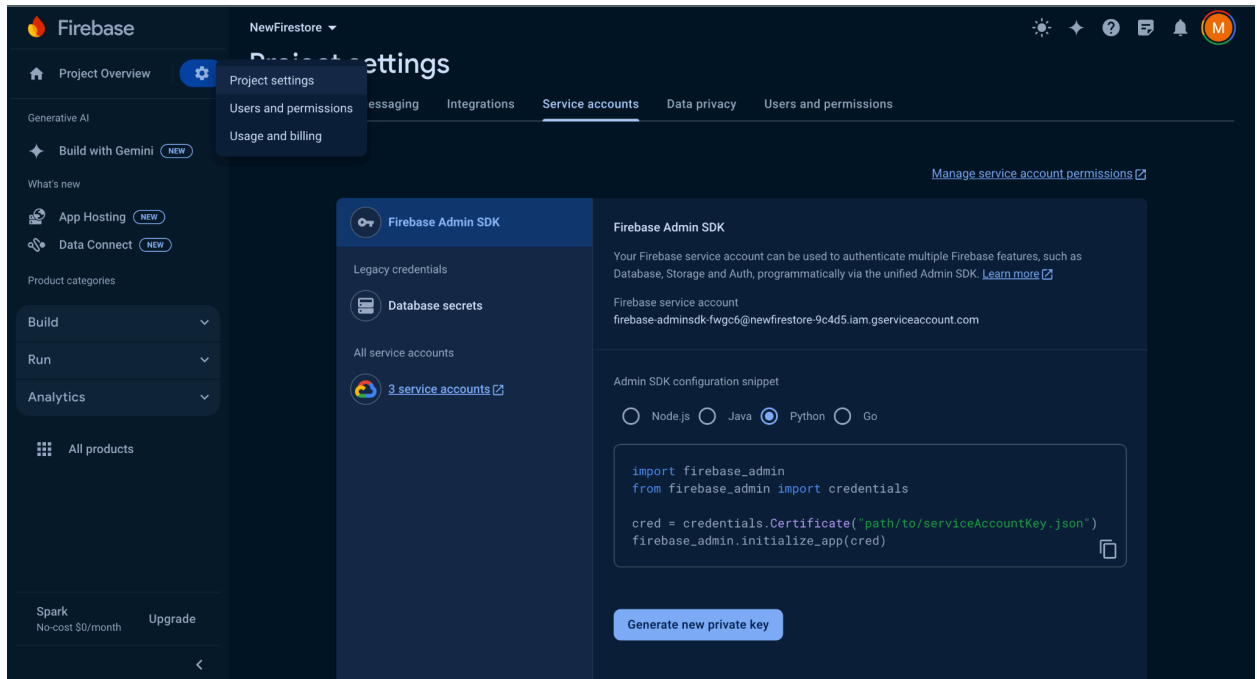


3. Connect the Firestore to the Python Application

In order to interface the Firestore Database with python, a secret key needs to be generated.

Navigate to: Project Overview → Project Settings → Service Accounts

Select Python and click 'Generate new private key', this will create a .json file



In the python code, ensure the following lines exist and are updated to reflect the new database.

```
import firebase_admin
from firebase_admin import credentials

cred = credentials.Certificate('path/to/serviceAccountKey.json')
app = firebase_admin.initialize_app(cred, {'databaseURL':
'https://url_to_fb.firebaseio.com'})
```

Helpful source:

<https://medium.com/theleanprogrammer/connecting-firebase-6102ef4eca08>

4. Helpful Notes

If data is not able to be written to firebase, a good place to check is the rules area. Sometimes, a rule is required to offer validation to the database. An easy rule to implement is a check on the current date. An example is as follows:

```
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4     match /{document=**} {
5       allow read, write: if
6         request.time < timestamp.date(2024, 12, 25);
7     }
8   }
9 }
```